

# DYNAMIC INVENTORY & PERFORMANCE

---

Cloud Discovery and  
Large-Scale Execution Strategies

*Professional Reference Documentation*

*Version 1.0*

# Table of Contents

---

1. Introduction
2. Dynamic Inventory: Concepts
  - 2.1 Static vs. Dynamic Inventory
  - 2.2 Inventory Plugins vs. Inventory Scripts
  - 2.3 How Inventory Plugins Work
3. Cloud Provider Inventory Plugins
  - 3.1 AWS EC2 (amazon.aws.aws\_ec2)
  - 3.2 Microsoft Azure (azure.azurecollection.azure\_rm)
  - 3.3 Google Cloud Platform (google.cloud.gcp\_compute)
  - 3.4 VMware vSphere (vmware.vmware.vms)
  - 3.5 Proxmox VE (community.general.proxmox)
4. Composing and Testing Dynamic Inventory
  - 4.1 Combining Dynamic and Static Sources
  - 4.2 Caching for Performance
  - 4.3 Testing with ansible-inventory
5. Performance Optimization
  - 5.1 SSH Pipelining
  - 5.2 Connection Multiplexing (ControlPersist)
  - 5.3 Forks and Parallelism
  - 5.4 Fact Caching
  - 5.5 Gathering = smart
6. Long-Running Tasks with async and poll
  - 6.1 The Problem with Blocking Tasks
  - 6.2 Async with Polling (poll > 0)
  - 6.3 Fire-and-Forget (poll = 0)
  - 6.4 Checking Results with async\_status
7. Execution Strategies
  - 7.1 Linear Strategy (Default)
  - 7.2 Free Strategy
  - 7.3 Serial: Batch Processing
  - 7.4 Strategy Comparison
8. Best Practices
9. Quick Reference

# 1. Introduction

---

Two forces dominate the operational reality of any serious Ansible deployment: infrastructure changes constantly, and deployments must complete in reasonable time even across thousands of hosts. This document addresses both with the tools Ansible provides.

- **Dynamic Inventory** — the mechanism by which Ansible discovers managed nodes directly from cloud provider APIs, hypervisors, and other sources of truth, eliminating the need to maintain static host lists by hand.
- **Performance Optimization & Execution Strategies** — the tunable parameters and execution models that determine how fast a playbook runs, how connections are reused, how long-running tasks are handled, and how tasks flow across a fleet of hosts.

Together they allow a single automation repository to target ephemeral cloud instances and to converge a large fleet efficiently, without the brittleness of static inventory or the sluggishness of default settings.

## 2. Dynamic Inventory: Concepts

---

### 2.1 Static vs. Dynamic Inventory

A static inventory is a file — INI or YAML — that lists hosts, groups, and variables. It is simple, version-controllable, and perfectly adequate when the infrastructure is stable. It becomes a liability when instances are created and destroyed regularly: the file drifts from reality, and engineers spend time updating it instead of automating.

A dynamic inventory is generated at runtime. Instead of a file, you supply a configuration that tells Ansible how to discover hosts from an external source. The inventory is always accurate because it is derived from the source of truth itself.

| Aspect      | Static Inventory                         | Dynamic Inventory                              |
|-------------|------------------------------------------|------------------------------------------------|
| Source      | A hand-maintained file.                  | Cloud API, hypervisor, CMDB, database.         |
| Freshness   | Stale as soon as infrastructure changes. | Always current.                                |
| Maintenance | Manual updates required.                 | None — generated at runtime.                   |
| Best for    | Stable bare metal, lab environments.     | Cloud, containers, ephemeral infrastructure.   |
| Format      | INI, YAML, TOML.                         | YAML configuration + plugin.                   |
| Variables   | Defined in the file.                     | Derived from provider metadata (tags, labels). |

*Table 1 — Static vs. dynamic inventory.*

### 2.2 Inventory Plugins vs. Inventory Scripts

Ansible supports two mechanisms for dynamic inventory. The older mechanism is an **inventory script**: an executable that outputs JSON in a specific format. The modern mechanism is an **inventory plugin**: a Python module integrated with Ansible's plugin system, configured through a YAML file.

- **Inventory plugins are the recommended approach.** They offer built-in caching, composability (a dynamic source can be combined with a static file), and active maintenance.
- **Inventory scripts are legacy.** They still work and are occasionally necessary for data sources with no existing plugin, but they must implement caching and output formatting themselves.

To see every inventory plugin available on your control node, including those provided by installed collections:

```
ansible-doc -t inventory -l
```

## 2.3 How Inventory Plugins Work

An inventory plugin is selected by the `plugin` key at the top of a YAML file. The filename usually ends in a suffix that identifies the plugin — for example, `aws_ec2.yml` or `azure_rm.yml` — but the `plugin:` key is what actually selects it.

A minimal plugin configuration has three parts:

- **plugin** — the fully qualified plugin name, such as `amazon.aws.aws_ec2`.
- **Source settings** — regions, subscriptions, projects, or filters that determine which hosts to discover.
- **Grouping and variable settings** — `keyed_groups`, `groups`, and `compose` that turn provider metadata into Ansible groups and host variables.

## 3. Cloud Provider Inventory Plugins

This section covers the five most common dynamic inventory sources: AWS EC2, Microsoft Azure, Google Cloud Platform, VMware vSphere, and Proxmox VE. In each case the pattern is the same: install the collection, configure credentials, write a YAML configuration file, and reference it as the inventory.

### 3.1 AWS EC2 (amazon.aws.aws\_ec2)

```
# Install the collection and the Python SDK
ansible-galaxy collection install amazon.aws
pip install boto3 botocore

# Configure AWS credentials (either environment variables or ~/.aws/credentials)
export AWS_ACCESS_KEY_ID="your-access-key"
export AWS_SECRET_ACCESS_KEY="your-secret-key"
export AWS_DEFAULT_REGION="us-east-1"
```

*Inventory configuration (inventory/aws\_ec2.yml):*

```
---
plugin: amazon.aws.aws_ec2

# Regions to query
regions:
  - us-east-1
  - eu-west-1

# Only include running instances
filters:
  instance-state-name: running
  "tag:Environment":
    - production
    - staging

# Create groups from instance metadata
keyed_groups:
  - key: tags.Environment
    prefix: env
    separator: "_"
  - key: instance_type
    prefix: type
  - key: placement.region
    prefix: region
  - key: tags.Role
    prefix: role

# Set connection variables from instance attributes
compose:
  ansible_host: private_ip_address
  ansible_user: "ubuntu"
  ansible_ssh_private_key_file: "~/.ssh/aws-key.pem"

# Explicit groups from tag values
groups:
```

```
webservers: tags.Role == 'web'
databases: tags.Role == 'database'
monitoring: tags.Role == 'monitoring'

# Optional caching
cache: true
cache_plugin: jsonfile
cache_connection: /tmp/aws_inventory_cache
cache_timeout: 300
```

### 3.2 Microsoft Azure (azure.azcollection.azure\_rm)

```
# Install the collection and the Azure SDK
ansible-galaxy collection install azure.azcollection
pip install -r ~/.ansible/collections/ansible_collections/azure/azcollection/requirements-azure.txt

# Authenticate -- either a service principal or Azure CLI credentials
export AZURE_SUBSCRIPTION_ID="your-subscription-id"
export AZURE_CLIENT_ID="your-client-id"
export AZURE_SECRET="your-client-secret"
export AZURE_TENANT="your-tenant-id"
```

#### *Inventory configuration (inventory/azure\_rm.yml):*

```
---
plugin: azure.azcollection.azure_rm

# Filter VMs by resource group and tags
include_vm_resource_groups:
  - ansible-inventory-test-rg
  - production-rg

exclude_host_filters:
  - powerstate != 'running'

# Group by tags, location, and OS
keyed_groups:
  - key: tags.Environment
    prefix: env
  - key: location
    prefix: location
  - key: os_profile.system
    prefix: os

# Host variables
hostvar_expressions:
  ansible_host: private_ip addresses[0]

# Conditional groups
conditional_groups:
  webservers: "'web' in (tags.Role | default(''))"
  databases: "'db' in (tags.Role | default(''))"

plain_host_names: true
```

### 3.3 Google Cloud Platform (google.cloud.gcp\_compute)

```
# Install the collection and the GCP SDK
ansible-galaxy collection install google.cloud
pip install google-auth requests

# Authenticate with a service account
export GCP_SERVICE_ACCOUNT_FILE="/path/to/service-account.json"
export GCP_PROJECT="your-project-id"
```

### ***Inventory configuration (inventory/gcp\_compute.yml):***

```
---
plugin: google.cloud.gcp_compute

projects:
  - your-project-id

auth_kind: serviceaccount
service_account_file: /path/to/service-account.json

# Filters -- only running instances
filters:
  - status = RUNNING

# Group by labels and zone
keyed_groups:
  - key: labels.environment
    prefix: env
    separator: "_"
  - key: zone
    prefix: zone
  - key: labels.role
    prefix: role

# Host variables
hostvar_expressions:
  ansible_host: networkInterfaces[0].networkIP

# Compose additional variables
compose:
  ansible_user: "'ubuntu'"

# Optional caching
cache: true
cache_plugin: jsonfile
cache_connection: /tmp/gcp_inventory_cache

cache_timeout: 300
```

## **3.4 VMware vSphere (vmware.vmware.vms)**

```
# Install the VMware collection
ansible-galaxy collection install vmware.vmware

# Credentials
export VMWARE_HOST="vcenter.example.com"
export VMWARE_USER="administrator@vsphere.local"
export VMWARE_PASSWORD="your-password"
```

**Inventory configuration (inventory/vmware.yml):**

```
---
plugin: vmware.vmware.vms

# vCenter connection
hostname: "{{ lookup('env', 'VMWARE_HOST') }}"
username: "{{ lookup('env', 'VMWARE_USER') }}"
password: "{{ lookup('env', 'VMWARE_PASSWORD') }}"
validate_certs: false

# Optional filters
properties:
  - name
  - guest.ipAddress
  - guest.hostName
  - config.name
  - runtime.powerState
  - guest.toolsRunningStatus

# Only powered-on VMs
filter:
  - runtime.powerState == "poweredOn"

# Group by VM attributes
keyed_groups:
  - key: config.name
    prefix: vm
  - key: guest.hostName
    prefix: host

# Host variables
compose:
  ansible_host: guest.ipAddress
  ansible_user: "ubuntu"
```

**3.5 Proxmox VE (community.general.proxmox)**

```
# Install the community.general collection (contains the Proxmox plugin)
ansible-galaxy collection install community.general

# Credentials -- token-based authentication is recommended
export PROXMOX_URL="https://proxmox.example.com:8006"
export PROXMOX_USER="ansible@pve"
export PROXMOX_TOKEN_ID="ansible"
export PROXMOX_TOKEN_SECRET="your-token-secret"
```

**Inventory configuration (inventory/proxmox.yml):**

```

---
plugin: community.general.proxmox

url: "{{ lookup('env', 'PROXMOX_URL') }}"
user: "{{ lookup('env', 'PROXMOX_USER') }}"
token_id: "{{ lookup('env', 'PROXMOX_TOKEN_ID') }}"
token_secret: "{{ lookup('env', 'PROXMOX_TOKEN_SECRET') }}"
validate_certs: false

# Only running VMs
want_facts: true
filters:
  - status == 'running'

# Group by VM type and node
keyed_groups:
  - key: proxmox_type
    prefix: type
  - key: proxmox_node
    prefix: node
  - key: tags
    prefix: tag

# Host variables
compose:
  ansible_host: proxmox_ipconfig0.ip | default(proxmox_net0.ip)

# Group by tag presence
groups:
  linux_vms: "'linux' in (tags | default([]))"
  windows_vms: "'windows' in (tags | default([]))"

# Caching
cache: true
cache_plugin: jsonfile

cache_connection: /tmp/proxmox_inventory_cache
cache_timeout: 300

```

**Note:** Proxmox VE does not have a dedicated collection under the `community.proxmox` namespace; the inventory plugin is provided by `community.general`. Check `ansible-doc -t inventory -l` for the exact plugin name in your installed version.

| Provider | Plugin (FQCN)                               | Collection                         | Credential source                        |
|----------|---------------------------------------------|------------------------------------|------------------------------------------|
| AWS EC2  | <code>amazon.aws.aws_ec2</code>             | <code>amazon.aws</code>            | Environment vars /<br>~/.aws/credentials |
| Azure    | <code>azure.azurecollection.azure_rm</code> | <code>azure.azurecollection</code> | Service principal / Azure CLI            |
| GCP      | <code>google.cloud.gcp_compute</code>       | <code>google.cloud</code>          | Service account JSON                     |
| VMware   | <code>vmware.vmware.vms</code>              | <code>vmware.vmware</code>         | vCenter username + password              |
| Proxmox  | <code>community.general.proxmox</code>      | <code>community.general</code>     | API token (recommended)                  |

Table 2 — Cloud provider inventory plugin summary.

## 4. Composing and Testing Dynamic Inventory

### 4.1 Combining Dynamic and Static Sources

An inventory directory can contain both static files and dynamic plugin configurations. Ansible merges them into a single inventory. This is useful when, for example, most hosts come from a cloud API but a few legacy systems are managed manually.

```
inventory/
|-- aws_ec2.yml           # dynamic -- AWS EC2 instances
|-- azure_rm.yml         # dynamic -- Azure VMs
|-- proxmox.yml          # dynamic -- Proxmox VMs
|-- static_hosts.yml     # static -- legacy systems
`-- group_vars/
    |-- all.yml
    `-- webservers.yml

# Reference the directory as the inventory source:
# ansible-playbook -i inventory/ site.yml
#
# Or set it in ansible.cfg:
# [defaults]
# inventory = ./inventory/
```

### 4.2 Caching for Performance

Every time an inventory plugin runs, it queries the external API. For large cloud accounts this can take several seconds. Caching stores the result for a configurable period so that repeated runs do not re-query the API.

```
---
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1

cache: true
cache_plugin: jsonfile
cache_connection: /tmp/aws_inventory_cache
cache_timeout: 300    # seconds

# To force a refresh on the next run, delete the cache:
# rm -rf /tmp/aws_inventory_cache
```

Caching is enabled per plugin, in the plugin's YAML configuration. The `cache_timeout` value determines how long a cached result remains valid.

### 4.3 Testing with `ansible-inventory`

Before running a playbook against a dynamic inventory, verify that the plugin discovers the hosts you expect and assigns the correct groups and variables.

```
# Render the group hierarchy as a tree
ansible-inventory -i inventory/aws_ec2.yml --graph

# Output:
# @all:
#   |--@env_production:
#   |   |--web-01
#   |   |--web-02
#   |   |--db-01
#   |--@env_staging:
#   |   |--staging-web-01
#   |--@role_web:
#   |   |--web-01
#   |   |--web-02
#   |--@type_t3_medium:
#   |   |--web-01
#   |   |--web-02

# Show every variable for a single host
ansible-inventory -i inventory/aws_ec2.yml --host web-01

# Dump the entire inventory as JSON
ansible-inventory -i inventory/aws_ec2.yml --list

# Verify connectivity to all discovered hosts
ansible all -i inventory/aws_ec2.yml -m ping
```

The `--graph` view is the fastest way to confirm that `keyed_groups` and `groups` have produced the intended structure.

## 5. Performance Optimization

Ansible is designed for correctness and idempotence, not for raw speed. Out of the box, it uses conservative defaults that are safe on any control node. On a modern machine managing hundreds or thousands of hosts, these defaults leave substantial performance on the table. This section covers the highest-impact tuning parameters.

### 5.1 SSH Pipelining

By default, Ansible copies a module file to the managed node, executes it, and removes it. SSH pipelining skips the file transfer step by piping the module content over the existing SSH connection. This reduces the number of network round trips per task and is generally safe to enable.

SSH pipelining is **disabled by default**. Enabling it is one of the single highest-impact changes you can make to a playbook's speed.

```
# ansible.cfg
[ssh_connection]
pipelining = True
```

**Caveat:** Pipelining requires that the remote user can run `sudo` without a TTY. If you encounter errors after enabling pipelining, add the following to the managed node's `sudoers` configuration:

```
# /etc/sudoers.d/ansible-pipelining
Defaults:ansible !requiretty
```

## 5.2 Connection Multiplexing (ControlPersist)

SSH multiplexing reuses a single TCP connection for multiple SSH sessions. Ansible enables it by default on modern systems, but the parameters are worth tuning for large fleets.

```
# ansible.cfg
[ssh_connection]
ssh_args = -o ControlMaster=auto -o ControlPersist=120s
control_path = /tmp/ansible-ssh-%h-%p-%r
```

ControlPersist=120s keeps the master connection open for two minutes after the last session closes. Subsequent tasks to the same host reuse that connection, eliminating the TCP handshake and SSH authentication cost.

## 5.3 Forks and Parallelism

Forks control how many hosts Ansible operates on in parallel. The default is 5, which is very conservative. Most control nodes can handle 50 or more, and large deployments often run 100–200 forks.

Increasing forks is the second major lever after pipelining. The optimum depends on the CPU, memory, and network bandwidth of the control node.

```
# ansible.cfg
[defaults]
forks = 50

# Or on the command line:
ansible-playbook site.yml -f 50

# Or from the environment:
export ANSIBLE_FORKS=50
```

Monitor the control node's CPU and memory while increasing forks. If either saturates, reduce the value. There is no benefit to setting forks higher than the number of hosts in the play.

## 5.4 Fact Caching

Fact gathering is expensive. On every play, Ansible runs the setup module on every host unless facts are already cached. For a fleet of 500 hosts, that can add minutes to every run.

Fact caching stores the gathered facts between runs. Combined with `gathering = smart`, Ansible reuses cached facts when available and re-gathers only when the cache has expired.

```
# ansible.cfg
[defaults]
gathering = smart
fact_caching = jsonfile
fact_caching_connection = /tmp/ansible_facts
fact_caching_timeout = 86400 # 24 hours
```

Other cache backends are available, including `redis` and `memcached`, which are useful when multiple control nodes share a cache.

## 5.5 Gathering = smart

The `gathering` setting in `ansible.cfg` controls when facts are collected:

- **implicit** (default) — facts are gathered at the start of every play.
- **explicit** — facts are never gathered automatically; the playbook must call `setup` or `gather_facts: true`.
- **smart** — facts are gathered only if they are not already cached. This is the recommended setting when fact caching is enabled.

For plays that do not need facts at all — for example, a play that only copies a static file — disable gathering entirely to save time:

```
- name: Deploy a static file
  hosts: all
  gather_facts: false
  tasks:
    - name: Copy the banner
      ansible.builtin.copy:
        src: banner.txt
        dest: /etc/issue
```

## 6. Long-Running Tasks with `async` and `poll`

### 6.1 The Problem with Blocking Tasks

By default, Ansible holds the connection to each managed node open until the task completes. This blocking behaviour is fine for short tasks, but it creates two problems for long-running operations:

- **Connection timeouts.** A task that runs longer than the SSH session allows will be terminated.
- **Wasted parallelism.** A fork is tied up waiting for the task to finish, reducing the number of other hosts that can be processed concurrently.

The `async` keyword addresses both by allowing the task to run in the background on the managed node.

### 6.2 Async with Polling (`poll > 0`)

With `poll` set to a positive number, Ansible starts the task in the background and then polls the managed node every `poll` seconds until the task completes or the `async` timeout is reached. This still blocks the next task, but the connection is not held open continuously, so long-running tasks do not trigger SSH timeouts.

```
- name: Run a long database migration
  ansible.builtin.command: /opt/myapp/bin/migrate --all
  async: 3600           # allow up to 1 hour
  poll: 30             # check every 30 seconds
  register: migration

- name: Show the result
  ansible.builtin.debug:
    msg: "Migration completed with rc={{ migration.rc }}"
```

**`async`** sets the maximum number of seconds the task is allowed to run. **`poll`** sets how frequently Ansible checks on the task. If the task finishes before the `async` timeout, Ansible picks up the result and moves on.

### 6.3 Fire-and-Forget (`poll = 0`)

With `poll` set to 0, Ansible starts the task in the background and immediately moves on to the next task without waiting. This is the "fire-and-forget" pattern: useful for operations that can run independently while other work proceeds.

```
- name: Start a background rebuild
  ansible.builtin.command: /opt/myapp/bin/rebuild-index
  async: 7200
  poll: 0
  register: rebuild_job

- name: Continue with other work while the rebuild runs
  ansible.builtin.debug:
    msg: "The rebuild is running in the background"
```

**Important:** When `poll` is 0, Ansible does not wait for the task, so a failure in the background job is not immediately surfaced. Use `async_status` later in the play to check whether it succeeded.

### 6.4 Checking Results with `async_status`

When a task is started with `poll: 0`, it returns a job ID. Use the `async_status` module to check on it.

```
- name: Start the background job
  ansible.builtin.command: /opt/myapp/bin/long-job
  async: 7200
  poll: 0
  register: job

- name: Continue with other tasks
  ansible.builtin.debug:
    msg: "Doing other work while the job runs"

- name: Wait for the background job to complete
  ansible.builtin.async_status:
    jid: "{{ job.ansible_job_id }}"
  register: job_result
  until: job_result.finished
  retries: 120
  delay: 30
```

The `until: job_result.finished` condition combined with `retries` and `delay` polls the job until it completes. This is the correct pattern for running many long tasks in parallel and then collecting all results.

## 7. Execution Strategies

A strategy determines how Ansible schedules tasks across the hosts in a play. The choice of strategy has a direct effect on both execution speed and the behaviour of the deployment.

### 7.1 Linear Strategy (Default)

In the **linear** strategy, Ansible executes each task on all hosts before moving on to the next task. The order is: task 1 on every host, then task 2 on every host, and so on. Hosts run in parallel up to the fork limit, but every host must finish task 1 before any host starts task 2.

Execution order (linear strategy):

```
Task 1 -> host1, host2, host3, ... (in parallel up to forks)
Task 2 -> host1, host2, host3, ...
Task 3 -> host1, host2, host3, ...
...
```

This is the default because it produces deterministic, easy-to-reason-about behaviour. If a task fails on one host, the other hosts are not left ahead of it. The trade-off is that a single slow host can hold up the entire play.

### 7.2 Free Strategy

In the **free** strategy, each host progresses through the task list as quickly as it can, independently of the other hosts. A fast host can complete the entire play while a slow host is still on task 2.

Execution order (free strategy):

```
host1: Task 1 -> Task 2 -> Task 3 -> ... -> end of play
host2: Task 1 -> Task 2 -> Task 3 -> ... -> end of play  (at its own pace)
host3: Task 1 -> Task 2 -> Task 3 -> ... -> end of play  (at its own pace)
```

***To enable the free strategy, set it at the play level:***

```
- name: Deploy the application
  hosts: appservers
  strategy: free
  tasks:
    - name: Install dependencies
      ansible.builtin.package:
        name: "{{ item }}"
        state: present
      loop: [git, curl, nginx]

    - name: Deploy the application
      ansible.builtin.git:
        repo: https://github.com/example/app.git
        dest: /opt/app

    - name: Restart the application
      ansible.builtin.service:
        name: app
        state: restarted
```

The free strategy is faster for heterogeneous fleets where hosts take different amounts of time to complete tasks. It is also less predictable: if you need every host to reach the same state before proceeding, linear is the safer choice.

## 7.3 Serial: Batch Processing

The serial keyword does not change the strategy; it changes how many hosts are included in each batch. With serial: 3, Ansible processes three hosts at a time. When all three have finished the entire play, the next three are processed, and so on.

```
- name: Rolling deployment
  hosts: appservers
  serial: 3          # process 3 hosts at a time
  tasks:
    - name: Deploy the new version
      ansible.builtin.unarchive:
        src: "myapp-{{ version }}.tar.gz"
        dest: /opt/myapp

    - name: Restart the application
      ansible.builtin.service:
        name: myapp
        state: restarted
```

serial can be a number, a percentage, or a list of batch sizes. The list form is useful for a gradual rollout:

```
- name: Gradual rollout
  hosts: appservers
  serial:
    - 1      # first batch: 1 host (canary)
    - 5      # second batch: 5 hosts
    - 25%    # remaining: 25% at a time
```

## 7.4 Strategy Comparison

| Aspect               | linear                                     | free                                            |
|----------------------|--------------------------------------------|-------------------------------------------------|
| Default              | Yes                                        | No                                              |
| Task order           | All hosts complete task N before task N+1. | Each host runs its own task list independently. |
| Speed                | Slower on heterogeneous fleets.            | Faster on heterogeneous fleets.                 |
| Predictability       | High — hosts stay in lockstep.             | Lower — hosts can diverge.                      |
| Best for             | Ordered deployments, dependency chains.    | Independent hosts, mixed-speed fleets.          |
| Failure handling     | A failed host is removed; others continue. | Same, but faster hosts may be far ahead.        |
| Combines with serial | Yes                                        | Yes (less meaningful).                          |

Table 3 — Linear vs. free strategy.

**Choosing a strategy:** Start with linear. It is the default for good reason. Switch to free only when you have measured that the linear strategy is the bottleneck — typically because a handful of hosts are much slower than the rest, or because tasks are independent and order does not matter.

## 8. Best Practices

---

### Dynamic Inventory

- **Use inventory plugins, not scripts.** Plugins are maintained, cached, and composable.
- **Always test with `--graph`.** Run `ansible-inventory --graph` after any change to a plugin configuration.
- **Cache aggressively.** Set a cache timeout appropriate to how often your infrastructure changes — 5 to 15 minutes is typical.
- **Pin the collection version.** Record the plugin's collection in `requirements.yml` with a version pin.
- **Use `keyed_groups` and `groups` deliberately.** Choose a naming scheme for group prefixes and stick to it.
- **Never store credentials in the inventory file.** Use environment variables, a credentials file, or a vault.
- **Combine with static sources when needed.** A dynamic source and a static file can coexist in the same inventory directory.

### Performance

- **Enable pipelining.** It is the single biggest win for most playbooks. Add `pipelining = True` to `ansible.cfg`.
- **Increase forks.** Raise the default of 5 to 20–50, or higher on a powerful control node. Monitor CPU and memory while you tune.
- **Enable fact caching.** Set `gathering = smart` and a JSON or Redis cache.
- **Disable gathering when facts are not needed.** Add `gather_facts: false` to plays that only copy files or run simple commands.
- **Tune `ControlPersist`.** Keep connections open for 120 seconds or more on stable networks.
- **Use `async` for long tasks.** Avoid SSH timeouts and free up forks with `async` and `poll: 0`.
- **Measure before and after.** Use `--diff` and the `profile_tasks` callback to identify where time is actually spent.

## 9. Quick Reference

---

### Dynamic Inventory

```
# AWS EC2
ansible-galaxy collection install amazon.aws
# inventory/aws_ec2.yml -> plugin: amazon.aws.aws_ec2

# Azure
ansible-galaxy collection install azure.azcollection
# inventory/azure_rm.yml -> plugin: azure.azcollection.azure_rm

# GCP
ansible-galaxy collection install google.cloud
# inventory/gcp_compute.yml -> plugin: google.cloud.gcp_compute

# VMware
ansible-galaxy collection install vmware.vmware
# inventory/vmware.yml -> plugin: vmware.vmware.vms

# Proxmox
ansible-galaxy collection install community.general
# inventory/proxmox.yml -> plugin: community.general.proxmox

# Test the inventory
ansible-inventory -i inventory/ --graph
ansible-inventory -i inventory/ --host myhost
```

## Performance tuning

```
# ansible.cfg -- high-impact settings
[defaults]
forks = 50
gathering = smart
fact_caching = jsonfile
fact_caching_connection = /tmp/ansible_facts
fact_caching_timeout = 86400

[ssh_connection]
pipelining = True
ssh_args = -o ControlMaster=auto -o ControlPersist=120s
control_path = /tmp/ansible-ssh-%%h-%%p-%%r
```

## Async and poll

```
# Long-running task with polling
- name: Long task
  ansible.builtin.command: /opt/app/long-job
  async: 3600
  poll: 30

# Fire-and-forget
- name: Background task
  ansible.builtin.command: /opt/app/long-job
  async: 7200
  poll: 0
  register: job

# Check the background job later
- name: Wait for completion
  ansible.builtin.async_status:
    jid: "{{ job.ansible_job_id }}"
  register: result
  until: result.finished
  retries: 120
  delay: 30
```

## Execution strategy

```
# Linear (default) -- all hosts complete each task before moving on
- hosts: all
  tasks: [...]

# Free -- each host progresses at its own pace
- hosts: all
  strategy: free
  tasks: [...]

# Serial -- process hosts in batches
- hosts: all
  serial: 3          # or 25% or a list of batch sizes
  tasks: [...]
```

## Useful commands

```
# List installed inventory plugins
ansible-doc -t inventory -l

# Inspect a dynamic inventory as a tree
ansible-inventory -i inventory/ --graph

# Show all variables for one host
ansible-inventory -i inventory/ --host web-01

# Increase forks on the command line
ansible-playbook site.yml -f 50

# Profile task execution time
ANSIBLE_CALLBACKS_ENABLED=profile_tasks ansible-playbook site.yml
```

*This document describes Ansible Core behaviour as of version 2.14 and later, and the cloud collection inventory plugins available at that time. Plugin names, configuration options, and performance defaults may evolve between releases; always consult the official documentation and ansible-doc for the version in use.*